You may think you're done, but there's one related issue to address now. You draw all the countries when the globe is first initialized, but many of them are clipped, and so your `geo.path.area()` function, which determines the area as the shape is drawn, has even worse issues than the Mercator projection had. For instance, in figure 7.13, Australia is colored as if it had an area similar to Madagascar. Fortunately, D3 also includes `d3.geo.area()`, which determines the spherical area of a shape corresponding to its geographic area, as in figure 7.14.

We could rewrite the draw code to use `d3.geo.area`, but instead let's recolor our existing globe. But how do we get the data? Until now, we've assumed that the data array was exposed somewhere our functions could get to, but what if it's outside our current scope? In this case, we can use `selectAll.data()` and get an array of data associated with whatever we select (which includes undefined elements if we select HTML elements that aren't bound with data). You'll see this in action more in the next chapter.

```
var featureData = d3.selectAll("path.countries").data();

var realFeatureSize =
d3.extent(featureData, function(d) {return d3.geo.area(d)});

var newFeatureColor =
d3.scale.quantize().domain(realFeatureSize).range(Reds[7]);

d3.selectAll("path.countries")
.style("fill", function(d) {return newFeatureColor(d3.geo.area(d))});
```

The spherical area of a shape as measured by `d3.geo.area()` is given in steradians, and so it's only a roughly proportionate area. If you want the actual square kilometers of a country or other shape, you'll still need to calculate that in a GIS package like QGIS, or get that information from another source.

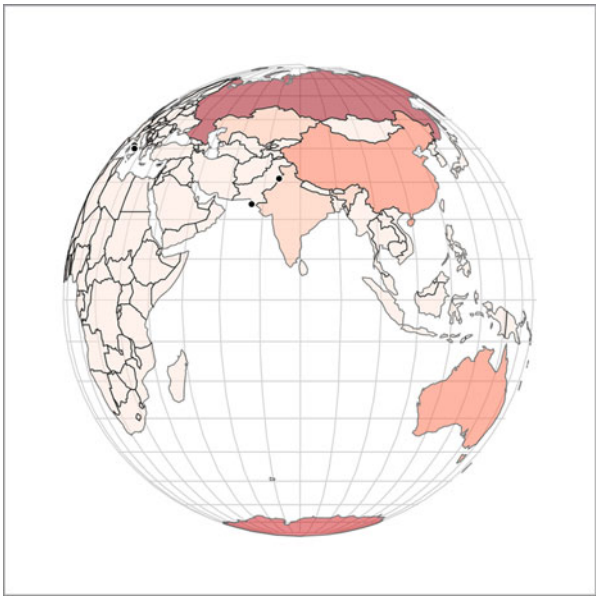


**Figure 7.14 Our globe with countries colored by their geographic area, rather than their graphical area**